

A3 LDA IMPLEMENTATION - Answers

LDA IMPLEMENTATION:

Implement Linear Discriminant Analysis (LDA) from scratch in Python without using any built-in LDA functions (e.g., `sklearn.discriminant_analysis.LinearDiscriminantAnalysis`).

Your implementation should:

1. Compute the class-wise means and the overall mean.
2. Construct the Within-Class Scatter Matrix (SW).
3. Construct the Between-Class Scatter Matrix (SB).
4. Solve the generalized eigenvalue problem for $SW^{-1}SB$.
5. Select the top k discriminant vectors based on the largest eigenvalues.
6. Project the given dataset onto the new lower-dimensional subspace.
7. Visualize the transformed data for a two-class or multi-class dataset.

Restrictions:

- You may use NumPy for matrix operations and Matplotlib for visualization.
- Do not use any built-in LDA implementation from Scikit-learn or other machine learning libraries.
- Clearly comment each step of your code and explain the role of SW, SB, eigenvalues, and eigenvectors in the LDA process.

Expected Output:

- Reduced-dimensional feature matrix.
- Plot showing the projected samples.
- Brief explanation of how LDA maximizes class separability.

Answer: Implementation uses labeled, reproducible synthetic two-class data because the PDF does not specify a dataset. S_W accumulates within-class deviations and S_B measures class-mean separation from the global mean. Eigenvectors of the regularized $S_W^{-1} S_B$ give discriminant directions; the largest eigenvalue corresponds to the strongest direction. Two classes permit at most one component. The projection maximizes class separation relative to within-class spread.

```
import numpy as np

def lda_fit_transform(X, y, n_components=None, ridge=1e-6):
    """Fit a regularized Fisher LDA projection and transform X."""
    X = np.asarray(X, dtype=float)
    y = np.asarray(y)
    classes = np.unique(y)
    if len(classes) < 2:
        raise ValueError("LDA requires at least two classes.")

    overall_mean = X.mean(axis=0)
    n_features = X.shape[1]
    within_scatter = np.zeros((n_features, n_features))
    between_scatter = np.zeros((n_features, n_features))
    class_means = {}

    for label in classes:
        class_samples = X[y == label]
        class_mean = class_samples.mean(axis=0)
        class_means[label] = class_mean
        centered = class_samples - class_mean
        within_scatter += centered.T @ centered
```

```

    mean_delta = (class_mean - overall_mean).reshape(-1, 1)
    between_scatter += len(class_samples) * (mean_delta @ mean_delta.T)

# Ridge regularization makes inversion/solving more stable when S_W is singular.
regularized_within = within_scatter + ridge * np.eye(n_features)
matrix = np.linalg.solve(regularized_within, between_scatter)
eigenvalues, eigenvectors = np.linalg.eig(matrix)
eigenvalues = np.real(eigenvalues)
eigenvectors = np.real(eigenvectors)
order = np.argsort(eigenvalues)[::-1]
max_components = min(n_features, len(classes) - 1)
if n_components is None:
    n_components = max_components
if not 1 <= n_components <= max_components:
    raise ValueError(f"n_components must be between 1 and {max_components}.")

directions = eigenvectors[:, order[:n_components]]
transformed = (X - overall_mean) @ directions
return transformed, directions, class_means, overall_mean, within_scatter, between_scatter, eigenvalues[order]

# The assignment does not provide sample data; use a reproducible synthetic example.
rng = np.random.default_rng(7)
class_0 = rng.normal(loc=[0.0, 0.0], scale=[0.8, 0.8], size=(30, 2))
class_1 = rng.normal(loc=[2.5, 2.5], scale=[0.8, 0.8], size=(30, 2))
X = np.vstack((class_0, class_1))
y = np.array([0] * len(class_0) + [1] * len(class_1))

X_lda, W, class_means, overall_mean, S_W, S_B, eigenvalues = lda_fit_transform(X, y, n_components=1)
print("Class means:\n", class_means)
print("Overall mean:", overall_mean)
print("Within-class scatter S_W:\n", S_W)
print("Between-class scatter S_B:\n", S_B)
print("Sorted eigenvalues:", eigenvalues)
print("Selected discriminant vector:\n", W)
print("Reduced feature matrix shape:", X_lda.shape)
print("First five projected samples:\n", X_lda[:5])

try:
    import matplotlib.pyplot as plt
    plt.scatter(X_lda[y == 0, 0], np.zeros((y == 0).sum()), label="Class 0", alpha=0.75)
    plt.scatter(X_lda[y == 1, 0], np.zeros((y == 1).sum()), label="Class 1", alpha=0.75)
    plt.xlabel("LD1")
    plt.yticks([])
    plt.title("Two classes projected onto the LDA direction")
    plt.legend()
    plt.grid(True, axis="x")
    plt.show()
except ImportError:
    print("Install matplotlib to show the projected-sample plot.")

```